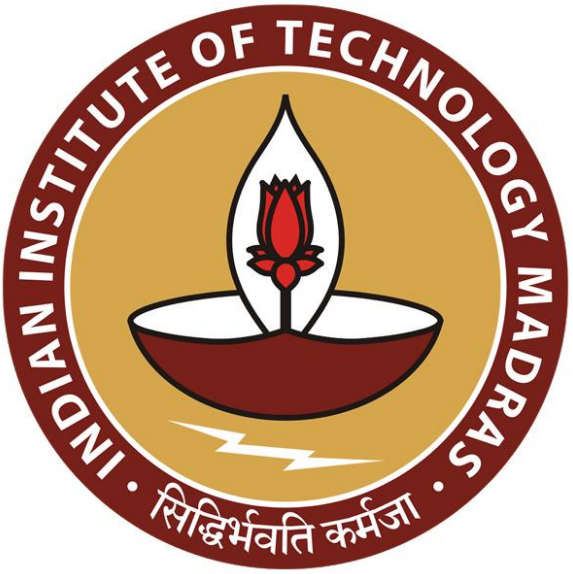


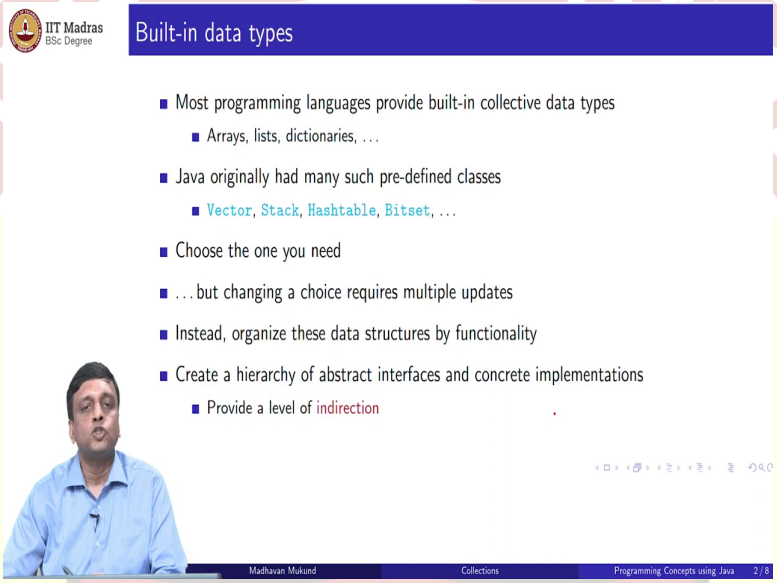
IIT Madras

ONLINE DEGREE

Programming Concepts using Java
Professor Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology, Madras
Collection

We saw that Java provides a bunch of data types. And we argued that these are organized in a particular way because of indirection.

(Refer Slide Time: 0:23)



So, many programming languages provide built in collective data types. So, if we have used Python, you know that you have lists and dictionaries. And if you use a library like NumPy, you have arrays, and so on. Python also has sets. So, each one of these provides a certain functionality. And depending on what the data structure is, you use it in a particular way.

So, for instance, in Python, if you have a dictionary, you can query its keys. But it does not make sense to query the keys of a list. If you have a list, you can access a value at a position. But it does not make sense to talk about the position normally in a dictionary. So, each data structure comes with its own pre specified list of operations, which are valid for it and depending on your choice, you go with that.

But this leaves us open to that same question that we raised when we were talking about the problem of indirection, which is that once you commit to one data structure, if you switch to another one, then usually there is a lot of work involved to make all your code switch. So,

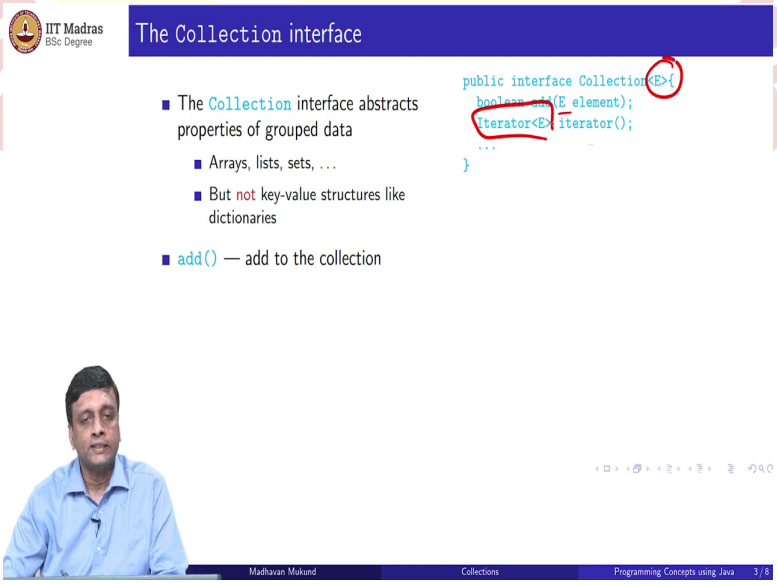
when Java was first designed, there were similarly a number of independent data structures, which were typically useful for programmers.

So, vector, which is a kind of a flexible array, stack which is Last In First Out structure, the last thing that you put in is the first thing that comes out. Hash tables, which we will talk about a bit, and so on. So, there were all these things in the same spirit, that for example, Python has lists and dictionaries, and sets, and so on. So, these are all different data structures, which offer different features in terms of usability and performance for storing different types of values.

So, you choose the one that you need, but changing your choice makes, requires you to make multiple updates. And that is exactly the problem that we are talking about. So, after some time, as Java was evolving, there was a decision to organize these data structures and provide a certain higher level kind of interlinking of these to make it more flexible to choose the one that you needed as late as possible, to add this possibility of indirection.

So, you create a hierarchy of abstract interfaces, and concrete implementation so that you can then switch around at the level of interfaces as far as possible, and choose a concrete implementation only when you need to. And if you need to change the concrete implementation, the interfaces can still be used, as we saw in the queue example. So, that the interfaces still remain the names of the various objects that you are dealing with. So, you do not have to change too much code.

(Refer Slide Time: 2:55)



So one such interface, which is the most commonly used one is called collection. So, collection encapsulate, or it subsumes all the things that we normally think of as collective data structures where we put a group of things, except for things like dictionary. So, in a dictionary, to get to a value, you have to provide a key remember, in a Python dictionary, you provide a key, and you get a value.

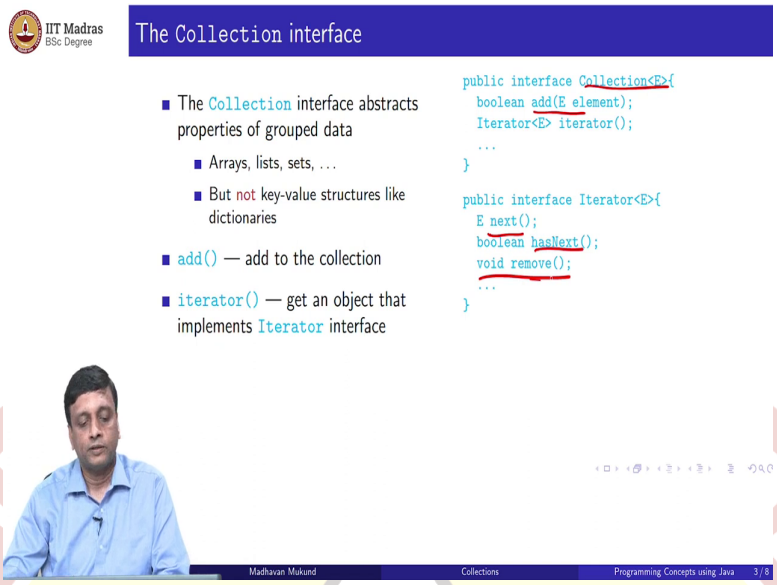
So, it is a each object that is stored in a dictionary has two parts, it has the key which tells you how to access it, and the value associated with the key. But in all other collections, like if I take a list or a set, or an array or something, I have a bunch of values, which may be separately organized in a sequence or as a collection without duplicates, or so on. But these are all groups of values. So, the collection interface subsumes all groups of values. But it is not for key value structures, which has a separate interface, which we will talk about later.

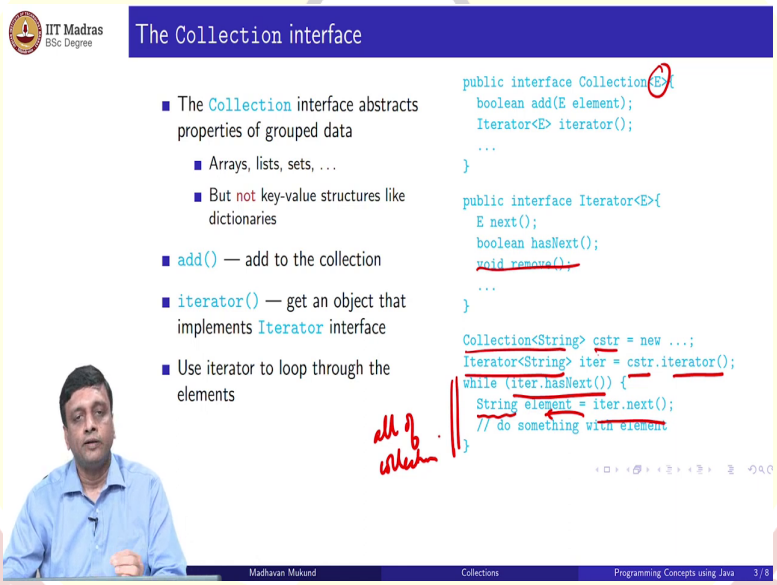
So, the most basic thing that you can do with a collection is you can add an element to it. So, add is a function which takes and so notice that we are using this generic thing that we saw last time, so where we have type variable, so we can have a collection over any underlying type. So, if I fixed the type, when I instantiate the collection, then add will take an element of that type and allow me to insert it into the collection. So, add is a very simple function.

An iterator we saw a long time back, we said that when we have these abstract things, and we need to interact with it and maintain multiple ways of running through it, the best way is to export an object which has the capability to walk through the data structure and give us one value at a time. And that is what we call an iterator.

So, in general collections have iterators. So, this is specified as a function which returns an iterator. And what it gives back is an object of type iterator. So, iterator is actually an interface which is sitting above collection, which is assumed to be available here. So, we have an iterator over Type E(Iterator<E> iterator();), which is what we get when we call this function iterator.

(Refer Slide Time: 5:02)





So, what is this interface of iterator promise us? It says that we have a function which can get us the next value. And in order to determine that is legal to get the next value, we have this query hasNext; which tells us whether or not there is one more value to get. Because we are trying to get a next value when there is no next value, then I am kind of it will throw an error because I have reached the end of my collection. So, before I get the next value, I must ask what is the next value.

In addition, it has something which we did not talk about earlier, we will come back to this later, which is that it has a remove operation. So, it also allows us to delete something. So, it is kind of important to note that this add operation is inside the collection. This remove

operation is with respect to an iterator. So, I have to first create an iterator and that iterator can remove I cannot directly ask the collection to remove something the iterator can remove, we will come back to this.

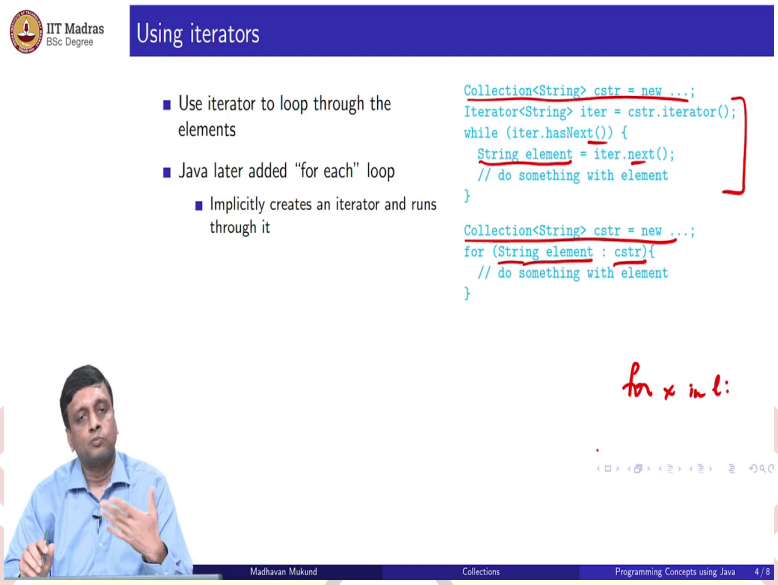
So, the usual use of an iterator if we ignore this remove part is to walk through an entire collection and process it. So, for instance, here I define a collection, I take a concrete value of E called string. So, I define something called cstr, which is a concrete collection of string and I instantiated with the appropriate thing and so on. And now I want to run through the cstr. So, what I do is I tell cstr to give me back an iterator and this will be of type iterator string because the cstr collection was created with a parameter string.

So, now I have this iterator called iter. So, what I will do is I will say that so long as this iter has something more to provide me so long as iter.hasNext()==True, I will get the next element and store it in a temporary variable of the type, which is the underlying collection in this case string.

So, I have this local variable element, which one by one will pick up these elements provided by this next function of the iterator and then I can do whatever I want with the element, I mean, I might want to inspect it, I want to check if it has certain value, I might be counting them, whatever it is, but I am guaranteed that this while loop will go to all of the collection. It will go through the elements of the collection systematically from beginning to end.

So, what we will see later is that is beginning to end may have different interpretations. And if the collection is a sequence, then it will go from the start to the end. If it is something like a set where there is no beginning and no end, all we are guaranteed is that it will give me all the values eventually, but not in any particular order. But the main purpose of an iterator is to run through the entire collection systematically and see every element in it using this next and hasNext.

(Refer Slide Time: 7:41)



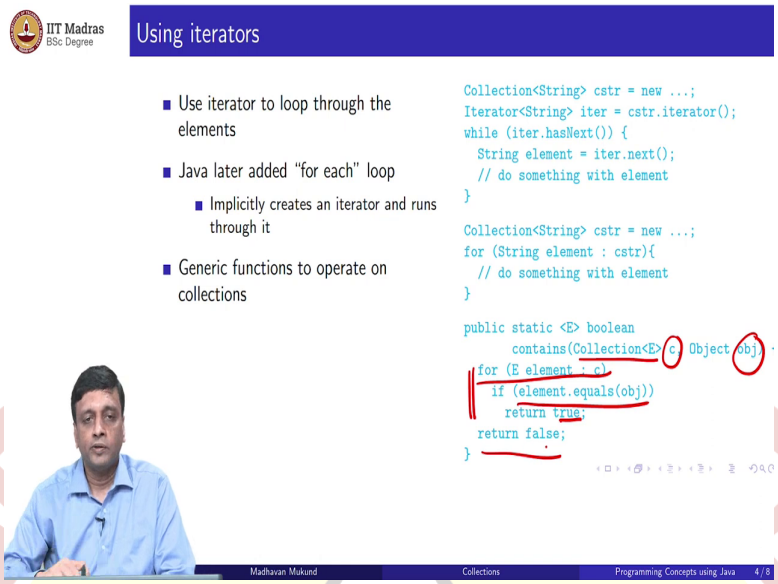
So, this is a very common thing. And in order to make it simpler for programmers to use this, something which we are now familiar with, with languages like Python, at some point, Java introduced this more elaborate, actually simpler, but from the point of view of the traditional for loop, a more elaborate for loop called a for each loop.

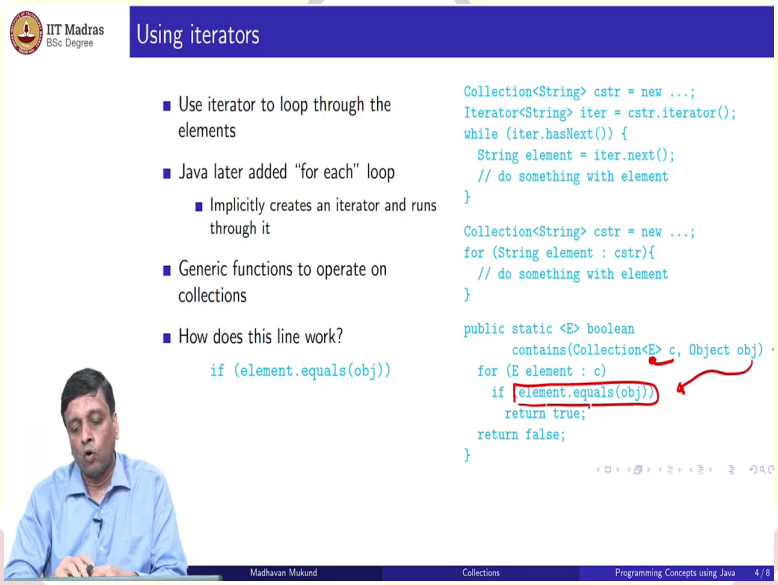
So, this for loop basically looks very simple. I create this collection as before and now instead of doing all this iterator business, I just run a for saying, I take this string element, and I make it a variable of my for loop. And I say this will now range over all the elements of my collection. So, this is just at one level, it is just what is called syntactic sugar, it is just a kind of a wrapper around an iterator.

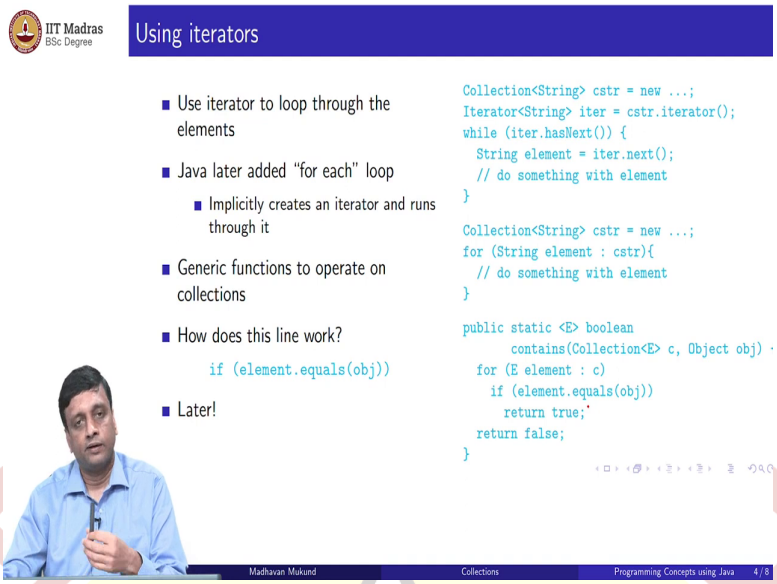
So, this for each loop implicitly creates an iterator and runs through it. But it does not explicitly give us this hasNext and next. We do not go through this hasNext, next thing, is just that this element will take first, the first position will take the value of the first element in the collection. The second element, pretty much like what we do in Python when we say for x in l.

So when we say for x in l, we just run through all elements in l from beginning to end. And then we just process each element as they are. So, this for each loop was introduced into Java and it basically captures the effect of the iterator with less syntax. There is no difference really between using a for and iterator, except the iterator requires more code to write.

(Refer Slide Time: 9:15)







So, now that I can run through my collection, I can also of course write generic functions. So, for instance, I can write this function which checks membership. I can ask whether in a collection there is an object? I, in a collection c is there an object there or not? So, I will go through all the elements in my collection. And for each of them I will check whether the element that I have just picked up from a collection equals the object or not.

If I find one, I will return true. If I go through this entire loop without finding any element that equals object, I will return false. Very straightforward. Nothing very complicated about it. Except perhaps this line which we will have to explain, and why is this line complicated? That is because this c is a collection of some arbitrary type E. And this is a kind of generic object.

So, we are kind of mixing two things here we are mixing the generic type that is associated with a parameterised data structure. And we are mixing this object, which is a kind of super type of everything. So, we have to kind of determine what this equals means, and we will do this as we go along.

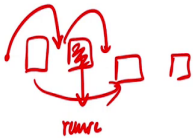
So, so far so good so what we have is we have this basic interface into of collection, which supports an add function, it supports an iterator with some very standard interface and using this iterator we can cycle through a collection. And we can do things like examine whether a value is equal to some other value or not.

(Refer Slide Time: 10:48)

Removing elements

- Iterator also has a `remove()` method
  - Which element does it remove?
- The element that was last accessed using `next()`

```
public interface Iterator<E>{
  E next();
  boolean hasNext();
  void remove();
  ...
}

Collection<String> cstr = new ...;
Iterator<String> iter = cstr.iterator();
while (iter.hasNext()) {
  String element = iter.next();
  // Delete element if it has some property
  if (property(element)) {
    iter.remove();
  }
}
```



Removing elements

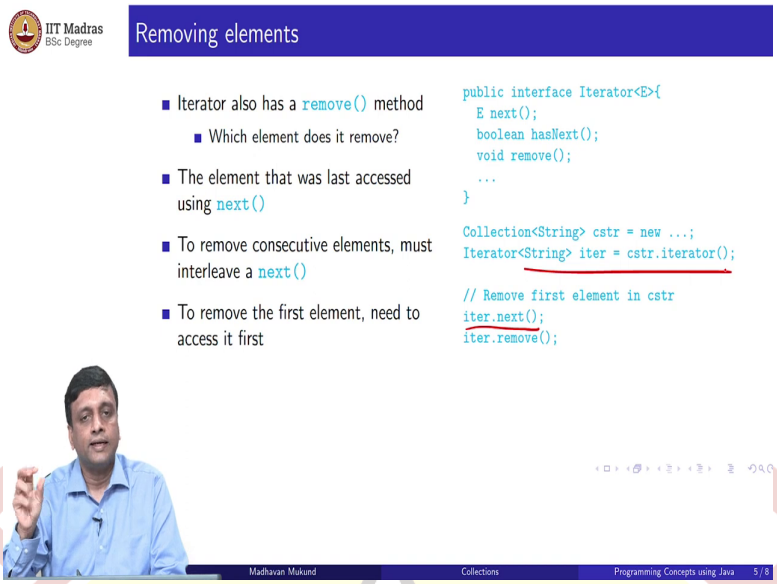
- Iterator also has a `remove()` method
  - Which element does it remove?
- The element that was last accessed using `next()`
- To remove consecutive elements, must interleave a `next()`

```
public interface Iterator<E>{
  E next();
  boolean hasNext();
  void remove();
  ...
}

Collection<String> cstr = new ...;
Iterator<String> iter = cstr.iterator();
...
iter.remove();
iter.remove(); // Error
```

Removing elements

- Iterator also has a `remove()` method
  - Which element does it remove?
- The element that was last accessed using `next()`
- To remove consecutive elements, must interleave a `next()`

```
public interface Iterator<E>{
  E next();
  boolean hasNext();
  void remove();
  ...
}

Collection<String> cstr = new ...;
Iterator<String> iter = cstr.iterator();
...
iter.remove();
iter.next();
iter.remove();
```



So, remember that this iterator also has a remove method. The remove method is in the iterator, is not in the underlying interface for collection. So, what is what is remove mean? So, the way that we go through a list or a collection is by using this iterator and using next. So, at any point next has returned some value to us. So, what remove does is that it removes that value.

So, if you think of collection this think of it as a sequence, so what we do is we start here, and the first time we do next, we come here, and this is available to us, the next time we do next we come here and this is available to us. So, at any given point, one of these values is available to us. And if I say remove, then this value will go. So, basically the collection will now short circuit from here to here. So the element that was accessed last, using the next is the one that will get removed.

So here for instance, we earlier said that we can go through a string. So, we will now do it explicitly a collection of strings. Now we will do it explicitly using an iterator. So, we get an iterator. And while we do hasNext, I get the next element, and I get the next element and I have it in my local variable element. So, now I can check whether this particular element, the value that I have just picked up from this collection, whether it satisfies some property or not.

So, property is some function that I have written somewhere else, which will tell me whether this element is to be removed or not. And if it has the property, then I just say, iter.remove(); And this implicitly removes the currently examined element, the element that I last picked up from next. So, that is how it works.

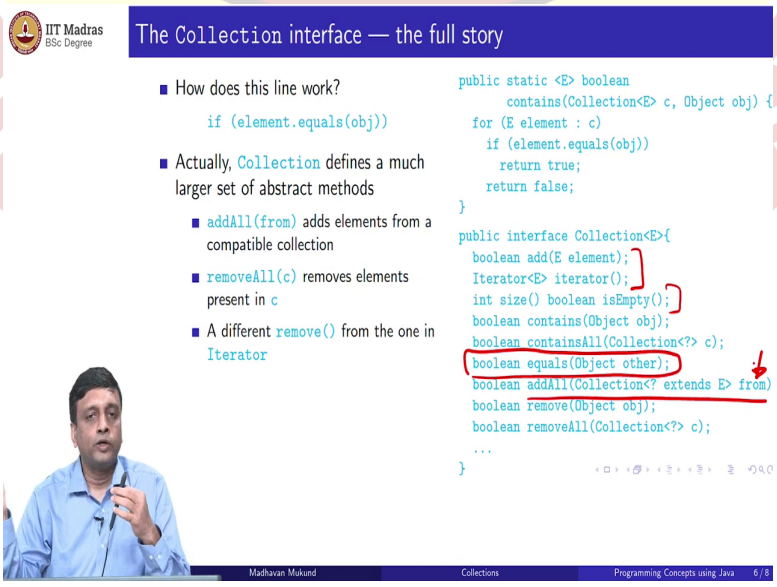
So, there is no explicit things saying remove this element or that element, you have to visit the element and the element that you last visited is implicitly the one that gets removed when you call remove. So, it is tied to the iterator. So, the iterator as it is going through, you can examine an element and choose to remove it.

So, this has some implications that is if I want to remove two elements in a row I must necessarily do a next. If I say remove and then follow it when other remove immediately this does not make sense because I just removed the element that I was looking at and there is no current element, in some sense iterator is in a kind of state where it is at a position where the value that it was it just passed over has been deleted. So, there is not I cannot remove it again.

So, I actually have to call a next and then remove it. So, every remove has to be preceded by a valid next. Only if that next is pointing to a value which has just been returned by the iterator, can I remove it. This also means that if I want to remove for instance the first element in a collection, so I have this collection of strings and I want to remove the first element, I will first have to access that first element.

So, I will have to start with an empty iterator, start a an initialize an iterator and then I will have to first say next to get the first element and then remove it. So, this is the thing that you have to remember that remove is tied to next you cannot do a remove without having done an access before.

(Refer Slide Time: 13:58)



The Collection interface — the full story

- How does this line work?

```
if (element.equals(obj))
```

- Actually, `Collection` defines a much larger set of abstract methods
  - `addAll(from)` adds elements from a compatible collection
  - `removeAll(c)` removes elements present in `c`
  - A different `remove()` from the one in `Iterator`

```
public static <E> boolean
      contains(Collection<E> c, Object obj) {
  for (E element : c)
    if (element.equals(obj))
      return true;
    return false;
}
public interface Collection<E>{
  boolean add(E element);
  Iterator<E> iterator();
  int size() boolean isEmpty();
  boolean contains(Object obj);
  boolean containsAll(Collection<?> c);
  boolean equals(Object other);
  boolean addAll(Collection<? extends E> from)
  boolean remove(Object obj);
  boolean removeAll(Collection<?> c);
  ...
}
```


The Collection interface — the full story

- How does this line work?

```
if (element.equals(obj))
```

- Actually, `Collection` defines a much larger set of abstract methods
  - `addAll(from)` adds elements from a compatible collection
  - `removeAll(c)` removes elements present in `c`
  - A different `remove()` from the one in `Iterator`

```
public static <E> boolean
      contains(Collection<E> c, Object obj) {
  for (E element : c)
    if (element.equals(obj))
      return true;
    return false;
}
public interface Collection<E>{
  boolean add(E element);
  Iterator<E> iterator();
  int size() boolean isEmpty();
  boolean contains(Object obj);
  boolean containsAll(Collection<?> c);
  boolean equals(Object other);
  boolean addAll(Collection<? extends E> from)
  boolean remove(Object obj);
  boolean removeAll(Collection<?> c);
  ...
}
```

Madhavan Mukund Collections Programming Concepts using Java 6 / 8

The Collection interface — the full story

- How does this line work?

```
if (element.equals(obj))
```

- Actually, `Collection` defines a much larger set of abstract methods
  - `addAll(from)` adds elements from a compatible collection
  - `removeAll(c)` removes elements present in `c`
  - A different `remove()` from the one in `Iterator`
- To implement the `Collection` interface, need to implement all these methods!

```
public static <E> boolean
      contains(Collection<E> c, Object obj) {
  for (E element : c)
    if (element.equals(obj))
      return true;
    return false;
}
public interface Collection<E>{
  boolean add(E element);
  Iterator<E> iterator();
  int size() boolean isEmpty();
  boolean contains(Object obj);
  boolean containsAll(Collection<?> c);
  boolean equals(Object other);
  boolean addAll(Collection<? extends E> from)
  boolean remove(Object obj);
  boolean removeAll(Collection<?> c);
  ...
}
```

Madhavan Mukund Collections Programming Concepts using Java 6 / 8

So, let us get back to this problematic line(element.equals(obj)). When we were doing this contains function this generic contains function, we said that we will examine this each element that we pick out of our collection and compare it to this object which we have as our argument, but one is of type capital E and the other one is a type Object. So, how does this work?

So, it turns out that what we saw earlier as the interface of collection is only the tip of the iceberg. So, in particular collection, we saw these things. And we probably saw size is also there, but it also has for instance this. So, the interface collection actually requires that you have a function which can sensibly compare an object of the underlying connection to any arbitrary incoming capital O object.

So, it is your duty to provide a meaningful function, because remember that if there is no meaningful function, then you will just use the pointer equality that is defined in capital O object, but you can choose to do casting, do whatever you want, and make it more meaningful equals but it is part of the requirement of the collection.

There are other things also, for instance, you have this addAll, so what does addAll do? It takes another collection, in this case, it is called from, it takes another collection, and it kind of imports all the values from. So, it takes the current collection and supplements it is like a union of two collections. Now, of course, if I have to take things from that collection and put it here, then this collection must be compatible with that collection in terms of type.

So, there is a constraint that so we, we now have this generic question mark, but it must extend the type of the current collection. It must be a subtype, only then can this type. So, remember, we did array copy, also, we said that the source type must be a subtype of the target type. So, here, this collection is receiving collections from the other one. So, this is the target. So, the source type must be a subtype of this. So, that is what it says.

removeAll, for instance, does something like kind of set difference. So, I have a collection, I give you another collection saying remove all these. So, for instance, I might have a list of, say, driving licenses, and then somebody gives me a list saying these driving licenses have expired. So, this is another collection of expired driving licenses. So, please remove all these driving licenses from your collection of valid driving licenses.

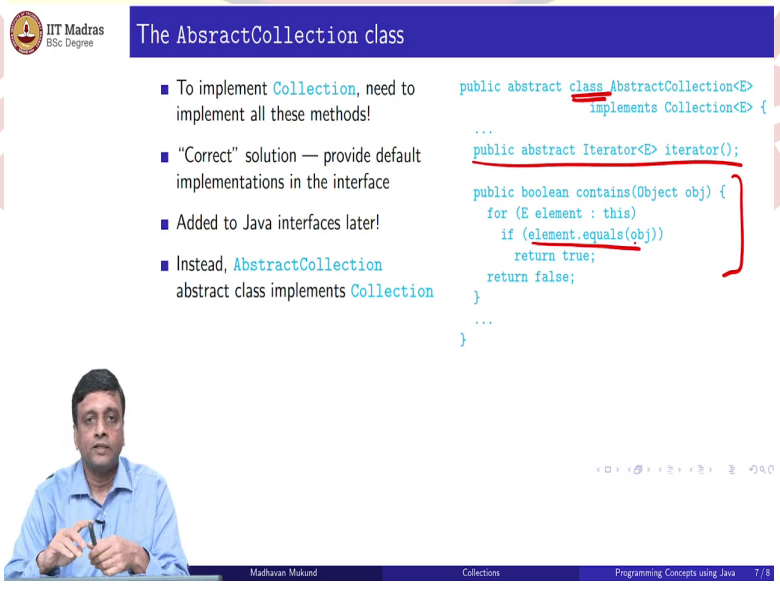
So, instead of doing it one by one, I can take that collection and do a kind of what is called a set difference, I can take this set and subtract that set retaining only those which are in this but not in that. Similarly, there is also something which will check whether this set is a superset. So, does my collection contain everything, which is there in the collection c?

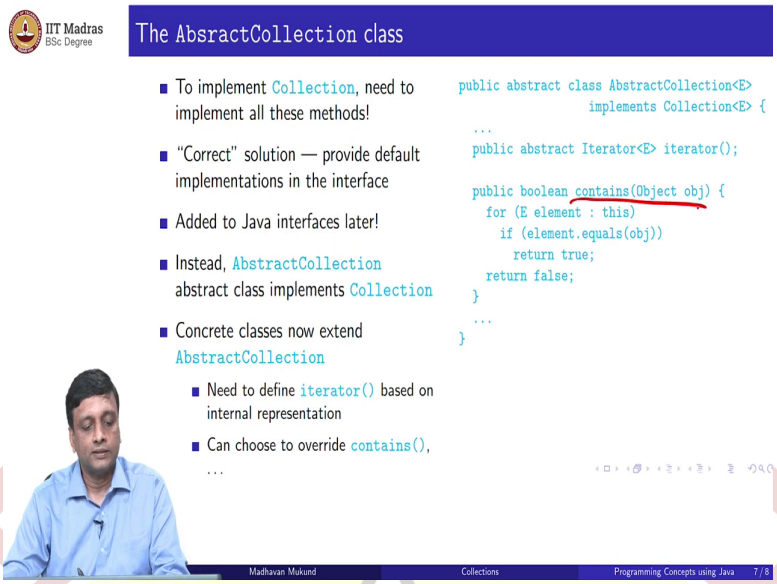
Now we talked about remove, so we said that the interface remove the interface of the iterator has remove, which removes a specific element that is being pointed to by the iterator at that point. But it turns out that this interface of collection also has remove. But this remove as a slightly different signature because it takes a specific object. So, it is saying if this object is there in your collection, remove it. So, this is different from the remove that we saw there, which said that implicitly remove the current object that the iterator is pointing.

So, of course, now, it is all very well to say that the collection is enhanced with all these extra requirements, which make these kinds of things meaningful in a generic method. But of course, for the downside of this is that you must if you want to create a concrete collection, you are supposed to implement everything.

So, as a interface, everything that is provided in the interface must be implemented. So, if you write something simple, like a list or an array or something, you have to write all these functions. And you have to figure out a way of importing and so on, which is a bit of, as you can imagine, a bit unsatisfactory and a bit of a nuisance.

(Refer Slide Time: 18:08)





So, what would we do? Well, remember that we said that interfaces are allowed to have default implementations. So, originally, the idea of an interface was something which had only abstract methods. But then we said that there may be situations where you might want to provide some default implementation so that you can implement the interface without having to concretely implement every function there, you can inherit some default implementations.

So, logically, that would have been the way to go to take collection, and then to not only define all these things, but maybe all these more exotic things, you might provide a default implementation so that you do not force the designer of an individual concrete collection class to actually implement everything. Well, the problem with this is that it is a question of how Java evolved.

Remember that we said by at the beginning, Java had these individual data structures like vector and hash set and all that, then there was a decision to group them into interfaces. So, the collection interface was designed and whatever we are going to talk about the collection interface came out as a result of that. But at this point, collections and the interfaces did not allow default methods. So, default methods came later.

So, at this point, in order to both allow this grouping of these collections into this interface, and yet not make it, onerous for the programmer, or very difficult for the programmer to do. The other solution was to actually provide an implementation through an abstract class. So, this is what Java has done. So, you have this collection, which is an interface. And then you have something called abstract collection, which is an abstract class.

It is a class and not an interface because at that particular point in time when this was designed and created an added to Java, Java interfaces could not implement functions. So, in this abstract class, some things remain abstract for instance, you remain, this iterator remains an abstract object because you cannot provide a concrete data structure and not write your own iterator because the iterator depends on the data structure.

So, you cannot rely on a default iterator because that default iterator will not in general, be able to handle your data structure. So, this is still an obligation for the designer of the data structure to implement. But this for instance, this contains function, which we wrote earlier, could be now provided as a generic contains function, that loop going through that we wrote as a generic function ourselves could actually be provided as a default implementation.

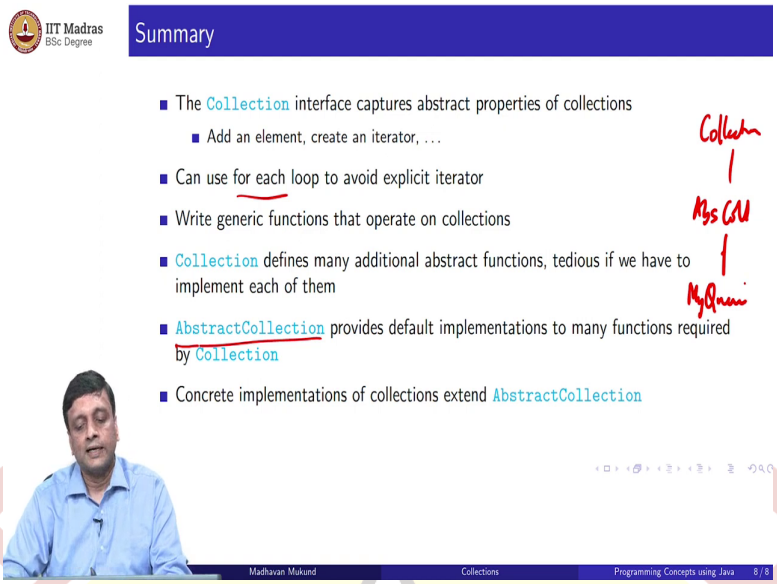
And this default implementation, notice, for instance, it uses equals, therefore, this presumably also a default implementation of equals. So, this abstract class provides us with several of the functions and it leaves us to focus only on the ones which we really normally would implement in terms of the interface, namely add, and the iterator and so on.

So, that is how this, the thinking behind this is that it should have been done possibly in current Java using default functions in the interpreter in the implementation of the interface itself. But, because it was not available at the time, when these interfaces were first designed, it is done through the second level called an abstract class.

So, what will actually happen is this abstract collection now becomes the starting point for any concrete collection. So, if you have to define now a real array or a real list or something, which you which extends this collection idea, you will not, you will not implement collection directly, because otherwise you have to implement all those functions, you will actually extend this abstract collection because then you have all these things which you can inherit., so you will inherit various things.

And of course, you can choose to overwrite for example, if you do not like this contains, which has come from abstract collection, you can write your own contains, but you do not have to write so this is the practical side of things.

(Refer Slide Time: 21:57)



So, the main point is that Java has reorganized all these collective data structures into this hierarchy of interfaces and abstract types. So, this is providing this indirection. So, collection in particular, the name collection stands for an interface which controls all these group types, which are not key value stores, which are not, which are not like dictionaries, or anything which is a list or an array or something is called a collection.

And this guarantees to us that you have functions to add an element to create an iterator and so on and using the iterator we can cycle through all the elements and then later on Java used this for each loop as a simplification or this iterator business. So, you can actually write what is now familiar to us as a Python style for loop to run through the elements in a collection.

We can also write generic functions because these are all now parameterize classes. So, they have this class variable as part of the definition. So, we can write generic functions that operate on many collections. And collection itself is quite an elaborate interface. It has many abstract functions, and it would be a real nuisance if we had to implement each of them every time we defined a data type.

So, what Java has done is it has created an intermediate abstract class called AbstractCollection, which actually gives default implementations for many things which are present in this interface. And the reason we have this two level thing is only because at the time when collection interfaces were defined, there was no possibility of adding default implementations in the interface in a modern setting, if you are redoing Java from scratch, we

can avoid AbstractCollection and just directly work with Collection and put all these template definitions are these default definitions into the class into the interface itself.

So therefore, now that we have this two level thing, when you actually have a concrete thing you will have Collection then you will have AbstractCollection. And then below this, you will have something like MyQueue. It will actually be an extension of the AbstractCollection and not directly implementing the Collection.